

MANUAL DE GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para desarrolladores y administradores en Debian GNU/Linux con Bash, GitHub CLI y GitHub Actions

[CLI & Web Workflows](#)[SSH & GPG Signing](#)[GitHub Actions CI/CD](#)[Issues & Projects v2](#)[Security & Dependabot](#)[Packages & Releases](#)[API & Scripting](#)

Manual de GitHub: De Novato a Avanzado en Debian Linux

Plataforma: Debian GNU/Linux (11 Bullseye / 12 Bookworm y derivadas como Ubuntu / Linux Mint)

Shell: Bash / GNU Coreutils

Herramientas: Git 2.40+, GitHub CLI (`gh`), OpenSSH, GnuPG, Libsecret

Índice de Contenidos

1. [Parte I: Fundamentos y Configuración del Entorno en Debian](#)
 - 1.1 [Diferencias entre Git y GitHub](#)
 - 1.2 [Instalación de Git y GitHub CLI \(`gh` \) en Debian](#)
 - 1.3 [Configuración de Identidad y Finales de Línea](#)
 - 1.4 [Autenticación Segura: SSH con Ed25519 y GitHub CLI](#)
 - 1.5 [Gestor de Credenciales en Linux \(`libsecret` \).](#)
 - 1.6 [Firma Criptográfica de Commits con GPG y SSH](#)
2. [Parte II: Flujo de Trabajo Esencial \(Nivel Novato\)](#)
 - 2.1 [Creación y Clonación de Repositorios](#)
 - 2.2 [El Ciclo de Tres Estados: Working Tree, Index y Commit](#)
 - 2.3 [Staging Selectivo y Convención de Commits](#)
 - 2.4 [Sincronización con el Repositorio Remoto](#)
 - 2.5 [Control de Archivos Ignorados \(`.gitignore` \).](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Ciclo de Vida de Ramas \(`git switch` , `git branch` \).](#)
 - 3.2 [Los Tres Métodos de Fusión en GitHub \(Merge, Squash, Rebase\).](#)
 - 3.3 [Modelos de Flujo de Trabajo: GitHub Flow, Git Flow y Forking](#)
 - 3.4 [Herramientas de Respaldo: `git stash` , `git cherry-pick` y `git rebase -i`](#)
 - 3.5 [El Salvavidas: Recuperar Commits y Ramas con `git reflog`](#)
4. [Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo](#)
 - 4.1 [Tema 1: Prevención y Buenas Prácticas de Equipo](#)
 - 4.2 [Tema 2: Fusión Automática \(Cambios en Distintas Líneas\)](#)
 - 4.3 [Tema 3: Conflicto Directo de Fusión \(Mismas Líneas\)](#)
 - 4.4 [Tema 4: Elección Total de Versión \(`--ours` vs `--theirs` \)](#)
 - 4.5 [Tema 5: Cambios Locales sin Confirmar al hacer Pull \(`git stash` \)](#)

- 4.6 [Tema 6: Push Rechazado por Desfase \(`non-fast-forward` \) y Rebase Seguro](#)
 - 4.7 [Tema 7: Resolución de Conflictos en Pull Requests \(Web y CLI\)](#)
 - 4.8 [Tema 8: Conflicto de Modificación vs Eliminación](#)
 - 4.9 [Tema 9: Conflictos en Archivos Binarios y Bloqueo con Git LFS](#)
5. [Parte V: Gestión de Proyectos y Ecosistema GitHub](#)
- 5.1 [GitHub Issues, Hitos y Etiquetas desde Terminal](#)
 - 5.2 [Pull Requests y Revisiones de Código desde el CLI](#)
 - 5.3 [GitHub Projects \(v2\): Tableros y Automatización](#)
 - 5.4 [GitHub Discussions y Wikis Locales](#)
6. [Parte VI: Automatización y CI/CD con GitHub Actions \(Nivel Avanzado\)](#)
- 6.1 [Estructura y Sintaxis de Workflows](#)
 - 6.2 [Pipelines para Debian/Linux: Tests, Linting y Matrices](#)
 - 6.3 [Secretos, Variables de Entorno y Caching](#)
 - 6.4 [Configuración de un Self-Hosted Runner en Debian como Servicio Systemd](#)
7. [Parte VII: Distribución, Paquetes y Publicación](#)
- 7.1 [GitHub Releases: Tags Semánticos y Binarios `.deb`](#)
 - 7.2 [GitHub Packages: Contenedores en GHCR](#)
 - 7.3 [GitHub Pages: Despliegue de Sitios Estáticos y Documentación](#)
8. [Parte VIII: Seguridad, Gobernanza y Políticas de Repositorio](#)
- 8.1 [Branch Protection Rules y Rulesets](#)
 - 8.2 [Dependabot, Secret Scanning y Push Protection](#)
 - 8.3 [Análisis Estático con CodeQL \(SAST\)](#)
 - 8.4 [Gobernanza con `CODEOWNERS` y Permisos](#)
9. [Parte IX: Scripting Avanzado con la API y Diagnóstico](#)
- 9.1 [Consultas a la API REST y GraphQL con `gh api`](#)
 - 9.2 [Configuración y Verificación de Webhooks](#)
 - 9.3 [Diagnóstico y Resolución de Problemas Frecuentes en Debian](#)
-

Parte I: Fundamentos y Configuración del Entorno en Debian

1.1 Diferencias entre Git y GitHub

- **Git:** Sistema de control de versiones distribuido (DVCS) creado por Linus Torvalds. Se ejecuta localmente en tu máquina Debian, almacena la historia en `.git` y funciona completamente sin conexión a internet.
- **GitHub:** Plataforma en la nube construida alrededor de Git. Añade alojamiento centralizado, colaboración en equipo mediante *Pull Requests*, automatización con *GitHub Actions*, seguimiento con *Issues*, registro de paquetes y seguridad de código.

1.2 Instalación de Git y GitHub CLI (`gh`) en Debian

Debian provee Git en sus repositorios oficiales, pero para interactuar con la plataforma en la nube requerimos GitHub CLI (`gh`).

Paso 1: Actualizar repositorios e instalar paquetes base

```
BASH
sudo apt update && sudo apt install -y curl wget git gnupg coreutils
```

¿Qué hace este comando?

Actualiza la lista de paquetes disponibles de Debian e instala herramientas indispensables para descargar claves seguras (`curl`, `wget`), el gestor de versiones (`git`) y el soporte de firmas criptográficas (`gnupg`).

Paso 2: Descargar el llavero oficial de GitHub CLI

```
BASH
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg | sudo tee
/etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
```

¿Qué hace este comando?

Crea el directorio de llaveros seguros de APT y descarga la clave pública oficial de GitHub para verificar criptográficamente que los paquetes que instalemos no hayan sido alterados.

Paso 3: Registrar el repositorio de GitHub CLI en APT

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-keyring.gpg] https://cli.github.com/packages stable main" | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
```

¿Qué hace este comando?

Añade el repositorio oficial de GitHub CLI adaptado automáticamente a la arquitectura de tu procesador Debian (`amd64`, `arm64`, etc.).

Paso 4: Instalar GitHub CLI y verificar versiones

```
sudo apt update && sudo apt install -y gh
git --version
gh --version
```

¿Qué hace este comando?

Descarga e instala el binario `gh` y valida que tanto Git como GitHub CLI estén correctamente instalados y listos para usar en la terminal.

1.3 Configuración de Identidad y Finales de Línea

Configurar nombre de autor y correo

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu-correo@ejemplo.com"
```

¿Qué hace este comando?

Establece la identidad global con la que se firmará la autoría de cada uno de tus commits.

Configurar saltos de línea y rama inicial

```
git config --global core.autocrlf input
git config --global init.defaultBranch main
git config --global core.editor nano
```

¿Qué hace este comando?

- `core.autocrlf input`: En Linux, asegura que los archivos se guarden en Git estrictamente con saltos de línea Unix **LF** (`\n`), convirtiendo cualquier CRLF que provenga de Windows.
- `init.defaultBranch main`: Establece `main` como el nombre predeterminado de la rama principal.
- `core.editor nano`: Configura el editor de texto interactivo para editar mensajes de commit.

1.4 Autenticación Segura: SSH con Ed25519 y GitHub CLI

GitHub no permite contraseñas por HTTPS; la autenticación debe realizarse mediante claves SSH o GitHub CLI.

Paso 1: Generar clave SSH Ed25519

```
ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com" -f ~/.ssh/id_ed25519
```

¿Qué hace este comando?

Crea un par de claves criptográficas modernas (privada y pública) mediante la curva elíptica Ed25519, más rápida y segura que RSA.

Paso 2: Iniciar el agente SSH y cargar la clave

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

¿Qué hace este comando?

Inicia el demonio de fondo `ssh-agent` y carga tu clave privada en memoria para que no tengas que ingresar la contraseña continuamente.

Paso 3: Configurar persistencia en ~/.ssh/config

```
BASH
cat << 'EOF' >> ~/.ssh/config
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
```

¿Qué hace este comando?

Indica al cliente SSH de Debian que utilice siempre tu clave `id_ed25519` al conectarse a `github.com` y fija permisos estrictos de lectura (`600`).

Paso 4: Iniciar sesión y sincronizar clave con GitHub CLI

```
BASH
gh auth login -p ssh -w
```

¿Qué hace este comando?

Inicia un asistente interactivo en el navegador web que autentica tu terminal con GitHub y asocia tu clave SSH pública automáticamente a tu cuenta.

Paso 5: Probar la conexión SSH

```
BASH
ssh -T git@github.com
```

¿Qué hace este comando?

Establece un canal SSH con GitHub para verificar el éxito de la autenticación (debe responder: `Hi usuario! You've successfully authenticated...`).

1.5 Gestor de Credenciales en Linux (libsecret)

```
sudo apt install -y libsecret-1-0 libsecret-1-dev build-essential
sudo make --directory=/usr/share/doc/git/contrib/credential/libsecret
git config --global credential.helper /usr/share/doc/git/contrib/credential/libsecret/git-credential-libsecret
```

¿Qué hace este comando?

Compila e instala el asistente de credenciales nativo de Linux para almacenar tokens de acceso personal de forma cifrada en el llavero de tu sesión.

1.6 Firma Criptográfica de Commits con GPG y SSH

GitHub destaca con la insignia **Verified** los commits firmados criptográficamente. Puedes usar directamente tu clave SSH para firmar:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

¿Qué hace este comando?

Configura Git para firmar digitalmente cada commit y tag utilizando tu clave pública SSH. Sube esta clave a GitHub bajo **Settings** → **SSH and GPG keys** → **New Signing Key**.

Parte II: Flujo de Trabajo Esencial (Nivel Novato)

2.1 Creación y Clonación de Repositorios

Caso A: Crear un repositorio local y publicarlo en GitHub

```
mkdir mi-proyecto-debian && cd mi-proyecto-debian
git init
echo "# Mi Proyecto en Debian" > README.md
git add README.md
git commit -m "docs: inicializar repositorio con README"
gh repo create mi-proyecto-debian --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Crea la carpeta, inicia el repositorio local (`git init`), genera el archivo `README.md` , crea el primer commit y utiliza `gh repo create` para crear el repositorio remoto en GitHub y subir los cambios inmediatamente.

Caso B: Clonar un repositorio existente

```
git clone git@github.com:usuario/mi-repositorio.git
```

¿Qué hace este comando?

Descarga la copia íntegra del repositorio remoto y su historial a tu máquina Debian a través de SSH.

2.2 El Ciclo de Tres Estados: Working Tree, Index y Commit

```
git status -s
```

¿Qué hace este comando?

Muestra el estado del árbol de trabajo de forma compacta (archivos modificados, añadidos o sin rastrear).

BASH

```
git diff
```

¿Qué hace este comando?

Compara los cambios presentes en tus archivos en disco contra el área de preparación (staging).

BASH

```
git diff --staged
```

¿Qué hace este comando?

Muestra las diferencias exactas de los cambios que ya están en el área de preparación y listos para ser confirmados.

2.3 Staging Selectivo y Convención de Commits

BASH

```
git add -p archivo.py
```

¿Qué hace este comando?

Abre el modo interactivo por bloques (hunks), permitiéndote seleccionar exactamente qué líneas de código enviar al staging y cuáles dejar pendientes.

Estándar Conventional Commits

Prefijo	Finalidad	Ejemplo
feat:	Incorpora una funcionalidad nueva	<code>git commit -m "feat: añadir endpoint REST de usuarios"</code>
fix:	Corrige un error o bug	<code>git commit -m "fix: solucionar pérdida de memoria en parser"</code>
docs:	Modifica documentación	<code>git commit -m "docs: documentar despliegue en Debian 12"</code>
refactor:	Refactoriza código sin cambio funcional	<code>git commit -m "refactor: optimizar bucle de procesamiento"</code>
test:	Añade o ajusta pruebas	<code>git commit -m "test: incorporar prueba unitaria de autenticación"</code>
chore:	Tareas rutinarias de configuración	<code>git commit -m "chore: actualizar librerías en Makefile"</code>

2.4 Sincronización con el Repositorio Remoto

```
git fetch origin
```

¿Qué hace este comando?

Descarga las ramas y commits nuevos desde GitHub a tu base de datos local sin tocar tus archivos de trabajo.

```
git pull --rebase origin main
```

¿Qué hace este comando?

Descarga los últimos cambios de `main` y reaplica tus commits locales encima de ellos, evitando commits de merge innecesarios.

```
git push -u origin main
```

¿Qué hace este comando?

Publica tus commits locales en la rama `main` de GitHub y asocia la rama para futuros envíos simples con solo escribir `git push`.

2.5 Control de Archivos Ignorados (.gitignore)

Crea un archivo `.gitignore` en la raíz del proyecto para evitar subir archivos no deseados:

```
PLAINTEXT

# Archivos temporales y de compilación
*.o
*.so
bin/
*.log

# Caches y entornos virtuales
__pycache__/
.venv/
.cache/
```

```
BASH

git rm --cached archivo_sensible.env
```

¿Qué hace este comando?

Remueve un archivo del control de versiones de Git sin borrar el archivo físico de tu disco.

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Ciclo de Vida de Ramas (`git switch`, `git branch`)

```
git switch -c feature/nueva-autenticacion
```

¿Qué hace este comando?

Crea una nueva rama llamada `feature/nueva-autenticacion` y conmuta el directorio de trabajo a ella en un solo paso.

```
git push -u origin feature/nueva-autenticacion
```

¿Qué hace este comando?

Publica la rama local en GitHub y establece el seguimiento remoto (*upstream tracking*).

```
git switch main
git branch -d feature/nueva-autenticacion
git push origin --delete feature/nueva-autenticacion
```

¿Qué hace este comando?

Vuelve a la rama principal, borra la rama local ya integrada (`-d`) y elimina la rama correspondiente en el servidor de GitHub.

3.2 Los Tres Métodos de Fusión en GitHub (Merge, Squash, Rebase)

- **Merge Commit** (`git merge --no-ff`): Une dos ramas preservando todos los commits individuales y crea un commit conmemorativo. Ideal cuando el historial individual de cada commit es valioso.
- **Squash and Merge**: Comprime todos los commits de la rama en un único commit limpio aplicado sobre la rama destino. Ideal para mantener un árbol principal impecable.

- **Rebase and Merge:** Aplica uno a uno los commits de la rama al final de `main` sin generar commit de unión. Proporciona una historia estrictamente lineal.

3.3 Modelos de Flujo de Trabajo: GitHub Flow, Git Flow y Forking

Forking Workflow para Proyectos de Código Abierto

```
BASH

# 1. Crear el fork y clonarlo localmente
gh repo fork organizacion/software-libre --clone
cd software-libre

# 2. Descargar actualizaciones del repositorio original (upstream)
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

¿Qué hace este comando?

Crea una copia en tu cuenta de GitHub, la descarga a tu máquina Debian y sincroniza tu rama `main` con las novedades del proyecto original.

3.4 Herramientas de Respaldo: `git stash`, `git cherry-pick` y `git rebase -i`

```
BASH

git stash save "Trabajo temporal en módulo de red"
```

¿Qué hace este comando?

Guarda tus cambios modificados y sin commitear en una pila temporal y deja tu directorio de trabajo completamente limpio.

```
BASH

git stash pop
```

¿Qué hace este comando?

Restaura el último conjunto de cambios guardados en el stash y lo elimina de la pila.

BASH

```
git cherry-pick 3a5b7c8
```

¿Qué hace este comando?

Aplica un commit específico (identificado por su hash) directamente sobre tu rama actual.

BASH

```
git rebase -i HEAD~3
```

¿Qué hace este comando?

Abre un editor interactivo para reescribir, combinar (squash), modificar el texto (reword) o eliminar los últimos 3 commits locales antes de enviarlos a GitHub.

3.5 El Salvavidas: Recuperar Commits y Ramas con `git reflog`

BASH

```
git reflog
```

¿Qué hace este comando?

Muestra un registro cronológico exhaustivo de cada cambio que sufrió el puntero `HEAD` en tu máquina (resets, checkouts, commits y rebases).

BASH

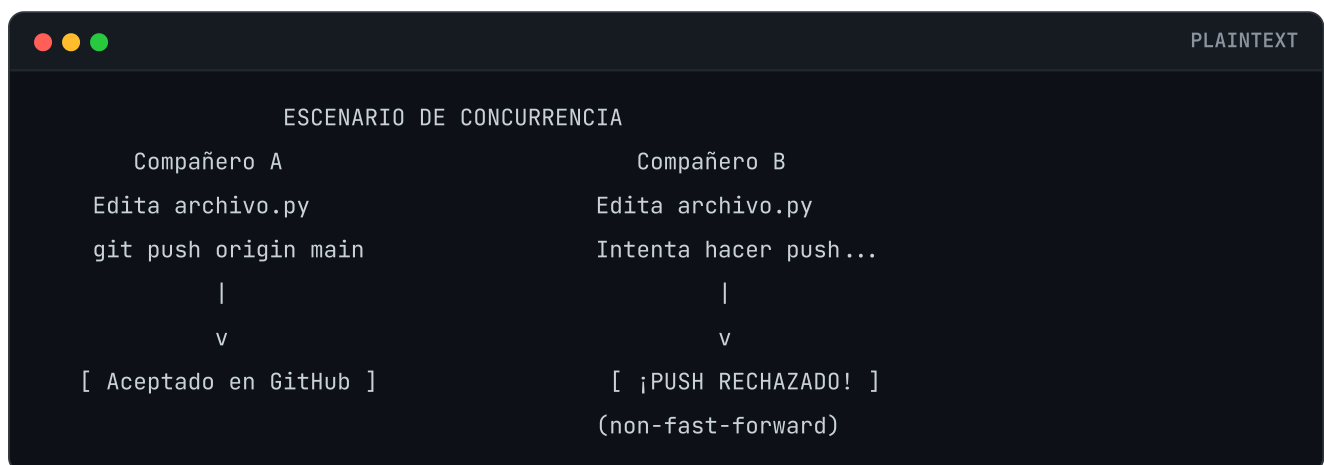
```
git reset --hard HEAD@{1}
```

¿Qué hace este comando?

Regresa el repositorio de forma exacta al estado previo a un error accidental (como un `git reset --hard` no deseado).

Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo

Uno de los desafíos más comunes en GitHub ocurre cuando **dos personas modifican el mismo archivo dentro de la misma carpeta**. A continuación se detallan todas las soluciones prácticas clasificadas por casos y temas:



4.1 Tema 1: Prevención y Buenas Prácticas de Equipo

La mejor forma de resolver un conflicto es evitar que ocurra mediante una organización estructurada:

1. **Nunca trabajar directamente sobre la rama `main`**: Cada miembro debe crear una rama específica para su tarea (`git switch -c feature/mi-modulo`).
2. **Modularización**: Dividir archivos gigantes en módulos más pequeños e independientes.
3. **Comunicación y Pull Requests tempranos**: Abrir un Pull Request en modo borrador (*Draft PR*) para que el equipo sepa qué archivos están siendo alterados.
4. **Sincronización frecuente**: Descargar los cambios de `main` con regularidad (`git pull --rebase origin main`) para no acumular semanas de desfase.

4.2 Tema 2: Fusión Automática (Cambios en Distintas Líneas)

Si tu compañero modificó el encabezado del archivo y tú editaste el final del mismo archivo, **Git es capaz de combinar los cambios de forma 100% automática**.

Procedimiento paso a paso:

```
BASH

# 1. Descargar e incorporar los cambios del compañero aplicando rebase
git pull --rebase origin main
```

¿Qué hace este comando?

Descarga el commit de tu compañero, coloca temporalmente tus commits en espera, avanza tu rama al estado del compañero y reaplica tus cambios. Al estar en líneas distintas, Git emite el mensaje:

```
Auto-merging src/archivo.py
```

```
Apply: feat: mis modificaciones
```

```
BASH

# 2. Verificar que el historial quedó limpio y lineal
git log --oneline -n 5

# 3. Enviar tus cambios a GitHub
git push origin main
```

4.3 Tema 3: Conflicto Directo de Fusión (Mismas Líneas)

Ocurre cuando ambos editaron exactamente las mismas líneas de código. Git detiene la operación e inserta delimitadores de conflicto en el archivo:

```
PYTHON

<<<<<< HEAD (Versión del repositorio / Tu compañero)
puerto_servicio = 8080
tiempo_espera = 30
=====
puerto_servicio = 9090
tiempo_espera = 60
>>>>>> feat: mi-cambio-local (Tu versión local)
```

Procedimiento de resolución paso a paso en Debian:

Paso 1: Identificar qué archivos están en conflicto

```
BASH
git status
```

¿Qué hace este comando?

Los archivos con conflicto aparecerán bajo el encabezado: `both modified: src/archivo.py`.

Paso 2: Abrir y editar el archivo en conflicto

```
BASH
nano src/archivo.py
```

¿Qué hace este comando?

Abre el archivo para edición manual. Debes:

1. Localizar los marcadores `<<<<<<<`, `=====` y `>>>>>>>`.
2. Dialogar o decidir cuál es el valor correcto (por ejemplo, dejar `puerto_servicio = 9090` y `tiempo_espera = 30`).
3. Borrar completamente las líneas con marcadores de conflicto dejando únicamente el código final limpio.
4. Guardar (`Ctrl + O`, `Enter`) y salir (`Ctrl + X`).

Paso 3: Marcar el conflicto como resuelto

```
BASH
git add src/archivo.py
```

¿Qué hace este comando?

Avisa a Git que el archivo ha sido reconciliado y preparado en el área de staging.

Paso 4: Concluir la operación

- Si estabas en un merge:

```
BASH
git commit -m "merge: resolver discrepancias en src/archivo.py"
git push origin main
```

- Si estabas en un rebase:

```
BASH
git rebase --continue
git push origin main
```

TIP

Si en cualquier momento te sientes desorientado y deseas cancelar la operación para volver al estado intacto anterior:

```
BASH
git merge --abort
# O bien:
git rebase --abort
```

4.4 Tema 4: Elección Total de Versión (--ours vs --theirs)

En ocasiones, una de las dos versiones es la correcta en su totalidad y no tiene sentido mezclar línea por línea.

Opción A: Conservar mi versión completa (descartar la del compañero)

```
BASH
# Durante un conflicto activo:
git checkout --ours src/archivo.py
git add src/archivo.py
git commit -m "resolve: conservar versión local de src/archivo.py"
git push origin main
```

¿Qué hace este comando?

Sobrescribe el archivo en conflicto exactamente con la versión que tú tenías, ignorando los cambios

remotos del compañero en ese archivo puntual.

Opción B: Aceptar la versión completa del compañero (descartar la mía)

```
# Durante un conflicto activo:
git checkout --theirs src/archivo.py
git add src/archivo.py
git commit -m "resolve: aceptar versión remota de src/archivo.py"
git push origin main
```

¿Qué hace este comando?

Reemplaza el archivo en conflicto con la versión que tu compañero subió a GitHub, descartando tus alteraciones locales en ese archivo.

4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull (`git stash`)

Si intentas hacer `git pull` mientras tienes cambios sin commitear en el mismo archivo que tu compañero subió, Git bloqueará la descarga con el error:

```
error: Your local changes to the following files would be overwritten by merge .
```

Procedimiento seguro con `git stash`:

```
# 1. Guardar tus cambios pendientes en el stash temporal
git stash save "Cambios locales en progreso"

# 2. Descargar los cambios que tu compañero subió
git pull --rebase origin main

# 3. Reaplicar tus cambios sobre la nueva base
git stash pop
```

¿Qué hace este comando?

- `git stash save` : Aparta tus modificaciones no guardadas y deja el directorio de trabajo limpio.
- `git pull --rebase` : Descarga con éxito las actualizaciones de GitHub.
- `git stash pop` : Vuelve a volcar tus cambios. Si coinciden en la misma línea, Git marcará el conflicto para que lo resuelvas con los pasos del Tema 3.

4.6 Tema 6: Push Rechazado por Desfase (non-fast-forward) y Rebase Seguro

Si hiciste commit localmente y al intentar hacer `git push` recibes:

```
! [rejected] main → main (fetch first) error: failed to push some refs
```

Significa que tu compañero subió un commit a GitHub antes que tú.

```
BASH

# 1. NUNCA fuerces con git push --force (destruirías el trabajo del compañero)

# 2. Traer los cambios del compañero y poner tus commits encima:
git pull --rebase origin main

# 3. Si no hay conflictos (o tras resolverlos con git rebase --continue):
git push origin main
```



CAUTION

Ejecutar `git push --force` sobrescribe el repositorio remoto con tu estado local, **borrando irrevocablemente los commits que tu compañero haya subido**. Si alguna vez requieres forzar una rama propia, utiliza siempre `git push --force-with-lease`, que se aborta si alguien más subió cambios que tú aún no has visto.

4.7 Tema 7: Resolución de Conflictos en Pull Requests (Web y CLI)

Cuando dos ramas abiertas en GitHub entran en conflicto sobre el mismo archivo:

Método A: Desde la interfaz Web de GitHub

1. Abre el Pull Request en GitHub.
2. Si existe conflicto, aparecerá el aviso: *This branch has conflicts that must be resolved*.
3. Haz clic en **Resolve conflicts**.
4. Edita el archivo directamente en el editor web eliminando los bloques `<<<<<<` y `>>>>>>`.
5. Haz clic en **Mark as resolved** y luego en **Commit merge**.

Método B: Desde la terminal con GitHub CLI (gh)

```
BASH

# 1. Descargar la rama del Pull Request a tu máquina
gh pr checkout 25

# 2. Incorporar los últimos cambios de main a la rama del PR
git fetch origin main
git merge origin/main

# 3. Resolver los conflictos en los archivos afectados
nano src/archivo.py
git add src/archivo.py
git commit -m "merge: resolver conflictos con rama main"

# 4. Subir la solución a GitHub (el PR se actualizará automáticamente)
git push origin HEAD
```

4.8 Tema 8: Conflicto de Modificación vs Eliminación

Ocurre cuando una persona modificó el archivo y la otra persona lo eliminó del repositorio.

Al hacer `git pull`, `git status` mostrará:

```
CONFLICT (modify/delete): src/archivo.py deleted in origin and modified in HEAD .
```

Opción 1: Deseas conservar el archivo (rechazar la eliminación)

```
BASH

git add src/archivo.py
git commit -m "resolve: conservar src/archivo.py frente a eliminación"
```

Opción 2: Deseas confirmar la eliminación (aceptar el borrado)

```
BASH

git rm src/archivo.py
git commit -m "resolve: confirmar eliminación de src/archivo.py"
```

4.9 Tema 9: Conflictos en Archivos Binarios y Bloqueo con Git LFS

Los archivos binarios (imágenes `.png`, archivos comprimidos `.zip`, ejecutables, bases de datos SQLite o documentos de diseño) no contienen texto plano y **no pueden fusionarse línea a línea**.

Caso A: Resolver conflicto binario eligiendo versión

```
BASH

# Quedarse con la imagen local
git checkout --ours assets/logo.png
git add assets/logo.png
git commit -m "resolve: conservar logo local"

# O quedarse con la imagen remota del compañero
git checkout --theirs assets/logo.png
git add assets/logo.png
git commit -m "resolve: aceptar logo remoto del compañero"
```

Caso B: Prevenir la edición concurrente con bloqueo (Git LFS Lock)

Instala Git LFS en Debian:

```
BASH

sudo apt install -y git-lfs && git lfs install
```

Configura un tipo de archivo para que soporte bloqueo exclusivo:

```
BASH

git lfs track "*.psd" --lockable
git add .gitattributes
git commit -m "chore: habilitar bloqueo exclusivo en archivos PSD"
git push origin main
```

Cuando vayas a editar el archivo, bloquéalo para que nadie más pueda subir cambios sobre él:

```
# Bloquear archivo antes de empezar a trabajar  
git lfs lock assets/disenio.psd
```

```
# Comprobar quién tiene bloqueado el archivo  
git lfs locks
```

```
# Tras terminar y hacer push, liberar el bloqueo  
git lfs unlock assets/disenio.psd
```

Parte V: Gestión de Proyectos y Ecosistema GitHub

5.1 GitHub Issues, Hitos y Etiquetas desde Terminal

```
gh issue create --title "Fallo en parseo de JSON" --body "El sistema falla con cadenas vacías." --label "bug,debian"
```

¿Qué hace este comando?

Crea una incidencia en GitHub con título, cuerpo y etiquetas sin salir de la terminal.

```
gh issue list --assignee "@me"
```

¿Qué hace este comando?

Lista todas las tareas e issues abiertos asignados a tu usuario.

```
gh issue close 42 --reason "completed"
```

¿Qué hace este comando?

Marca el issue número 42 como resuelto y lo cierra formalmente.

5.2 Pull Requests y Revisiones de Código desde el CLI

```
gh pr create --title "feat: integración con systemd" --body "Automatiza el arranque del servicio." --draft
```

¿Qué hace este comando?

Abre un nuevo Pull Request en modo borrador (Draft) desde tu rama actual hacia `main`.

BASH

```
gh pr checkout 15
```

¿Qué hace este comando?

Descarga automáticamente la rama del PR #15 a tu máquina Debian para que puedas compilarla y probarla localmente.

BASH

```
gh pr review 15 --approve -b "Validado en Debian 12 sin fallos."
```

¿Qué hace este comando?

Emite una aprobación oficial en el Pull Request con un comentario de verificación.

5.3 GitHub Projects (v2): Tableros y Automatización

BASH

```
gh project list
```

¿Qué hace este comando?

Lista los proyectos y tableros interactivos asociados a tu cuenta u organización.

BASH

```
gh project item-add 3 --owner "mi-organizacion" --url  
"https://github.com/usuario/repo/issues/42"
```

¿Qué hace este comando?

Vincula el issue #42 al tablero Kanban del proyecto #3.

5.4 GitHub Discussions y Wikis Locales

```
git clone git@github.com:usuario/mi-proyecto-debian.wiki.git
cd mi-proyecto-debian.wiki
echo "## Especificaciones de Arquitectura" >> Home.md
git commit -am "docs: expandir wiki técnica"
git push origin master
```

¿Qué hace este comando?

Clona la documentación wiki de GitHub como un repositorio Git local, añade contenido y sincroniza los cambios en la nube.

Parte VI: Automatización y CI/CD con GitHub Actions (Nivel Avanzado)

6.1 Estructura y Sintaxis de Workflows

Los archivos de flujo de trabajo se alojan en `.github/workflows/*.yml` y se ejecutan ante eventos como `push` o `pull_request`.

6.2 Pipelines para Debian/Linux: Tests, Linting y Matrices

Crea el archivo `.github/workflows/debian-ci.yml`:

```

name: Debian Linux CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]

    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v4

      - name: Configurar entorno Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
          cache: 'pip'

      - name: Instalar paquetes del sistema y dependencias
        run: |
          sudo apt update && sudo apt install -y build-essential libpq-dev
          pip install --upgrade pip
          if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
          pip install flake8 pytest

      - name: Ejecutar análisis estático (Linting)
        run: |
          flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics

      - name: Ejecutar pruebas unitarias
        run: |
          pytest --junitxml=reports/test-results-${ matrix.python-version }.xml

      - name: Subir reporte de pruebas como artefacto

```

```
uses: actions/upload-artifact@v4
if: always()
with:
  name: test-results-${{ matrix.python-version }}
  path: reports/
```

6.3 Secretos, Variables de Entorno y Caching

Configura tus credenciales en **Settings** → **Secrets and variables** → **Actions** y accede a ellas de forma cifrada:



```
- name: Despliegue seguro a servidor Debian
  env:
    SSH_KEY: ${ secrets.PROD_SERVER_SSH_KEY }
    HOST: ${ vars.PROD_HOST_IP }
  run: |
    echo "$SSH_KEY" > key.pem && chmod 600 key.pem
    ssh -i key.pem -o StrictHostKeyChecking=no admin@$HOST "systemctl restart mi-servicio"
```

6.4 Configuración de un Self-Hosted Runner en Debian como Servicio Systemd

Si cuentas con tu propio servidor físico o VPS Debian para ejecutar pipelines privados:

```
# 1. Crear usuario dedicado y directorio
sudo adduser --disabled-password --gecos "" actions-runner
sudo usermod -aG docker actions-runner
sudo su - actions-runner
mkdir actions-runner && cd actions-runner

# 2. Descargar el binario oficial del runner para Linux x64
curl -o actions-runner-linux-x64-2.316.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.316.0/actions-runner-linux-x64-
2.316.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.316.0.tar.gz

# 3. Registrar con el token provisto por GitHub
./config.sh --url https://github.com/usuario/repo --token TU_TOKEN_AQUI

# 4. Instalar y arrancar como servicio del sistema (Systemd)
sudo ./svc.sh install actions-runner
sudo ./svc.sh start
sudo ./svc.sh status
```

Parte VII: Distribución, Paquetes y Publicación

7.1 GitHub Releases: Tags Semánticos y Binarios `.deb`

```
# 1. Crear y subir un tag firmado
git tag -a v1.0.0 -m "release: versión 1.0.0 estable"
git push origin v1.0.0

# 2. Publicar la Release y adjuntar el paquete instalable de Debian
gh release create v1.0.0 ./dist/mi-app_1.0.0_amd64.deb \
  --title "Versión 1.0.0 para Debian" \
  --generate-notes
```

¿Qué hace este comando?

Crea una publicación oficial en GitHub vinculada al tag `v1.0.0`, genera automáticamente las notas de la versión a partir de los PRs fusionados y adjunta el paquete instalable `.deb`.

7.2 GitHub Packages: Contenedores en GHCR

```
# Iniciar sesión en el registro con un PAT con alcance 'write:packages'
echo $CR_PAT | docker login ghcr.io -u TU_USUARIO --password-stdin

# Construir y subir imagen Docker
docker build -t ghcr.io/tu-usuario/mi-app:1.0.0 .
docker push ghcr.io/tu-usuario/mi-app:1.0.0
```

7.3 GitHub Pages: Despliegue de Sitios Estáticos y Documentación

Configura `.github/workflows/pages.yml` para desplegar portales generados (por ejemplo con MkDocs, VitePress o Astro):

```
name: Desplegar a GitHub Pages

on:
  push:
    branches: [ main ]

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: false

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: './site'
      - id: deployment
        uses: actions/deploy-pages@v4
```

Parte VIII: Seguridad, Gobernanza y Políticas de Repositorio

8.1 Branch Protection Rules y Rulesets

Desde **Settings** → **Rules** → **Rulesets**:

- **Require a pull request before merging:** Impide que cualquier colaborador suba cambios directos a `main`.
- **Require approvals:** Exige que al menos un revisor apruebe el código.
- **Require status checks to pass:** Garantiza que los tests de GitHub Actions terminen en verde antes de permitir la fusión.
- **Block force pushes:** Prohíbe reescribir la historia remota con `push --force`.

8.2 Dependabot, Secret Scanning y Push Protection

Crea el archivo `.github/dependabot.yml`:

```
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
```

IMPORTANT

Push Protection viene activo por defecto. Si intentas hacer commit de una clave privada, token de API o contraseña, GitHub rechazará el `git push` en el servidor antes de recibir los datos.

8.3 Análisis Estático con CodeQL (SAST)

Activa el análisis de seguridad desde **Settings** → **Code security and analysis** → **CodeQL analysis**. CodeQL inspecciona cada Pull Request buscando fallos de seguridad (inyecciones SQL, problemas de memoria, validaciones nulas) antes de pasar a producción.

8.4 Gobernanza con CODEOWNERS y Permisos

Crea `.github/CODEOWNERS` :

A screenshot of a terminal window with a dark background. The window has three colored window control buttons (red, yellow, green) in the top-left corner and the text "PLAINTEXT" in the top-right corner. The terminal displays the content of the `.github/CODEOWNERS` file, which is a text file with comments and permissions for different parts of the repository.

```
# Mantenimiento global
* @mi-organizacion/tech-leads

# Empaquetado Debian
/debian/ @usuario-debian-maintainer
Makefile @usuario-debian-maintainer

# Workflows de CI/CD
/.github/workflows/ @mi-organizacion/devops-team
```

Parte IX: Scripting Avanzado con la API y Diagnóstico

9.1 Consultas a la API REST y GraphQL con `gh api`

```
gh api user/repos --paginate | jq '[][.name']
```

¿Qué hace este comando?

Consulta la API REST de GitHub para obtener todos los repositorios de tu cuenta paginados y extrae solo sus nombres con `jq`.

```
gh api --method POST repos/:owner/:repo/labels \
  -f name="debian-package" \
  -f color="d70a53" \
  -f description="Tareas sobre empaquetado Debian"
```

¿Qué hace este comando?

Crea una nueva etiqueta personalizada directamente en el repositorio remoto usando el método HTTP POST.

9.2 Configuración y Verificación de Webhooks

Script receptor en Python (Debian) con validación criptográfica HMAC-SHA256:

```

import hmac
import hashlib
from flask import Flask, request, abort

app = Flask(__name__)
SECRET = b"mi_clave_secreta_webhook"

@app.route("/webhook", methods=["POST"])
def github_webhook():
    signature_header = request.headers.get("X-Hub-Signature-256")
    if not signature_header:
        abort(403)

    hash_type, signature = signature_header.split("=")
    computed = hmac.new(SECRET, request.data, hashlib.sha256).hexdigest()

    if not hmac.compare_digest(signature, computed):
        abort(403)

    event = request.headers.get("X-GitHub-Event")
    print(f"Evento recibido de GitHub: {event}")
    return "OK", 200

if __name__ == "__main__":
    app.run(port=5000)

```

9.3 Diagnóstico y Resolución de Problemas Frecuentes en Debian

1. Permission denied (publickey)

- **Solución:** Comprueba que tu clave SSH esté cargada y validada:

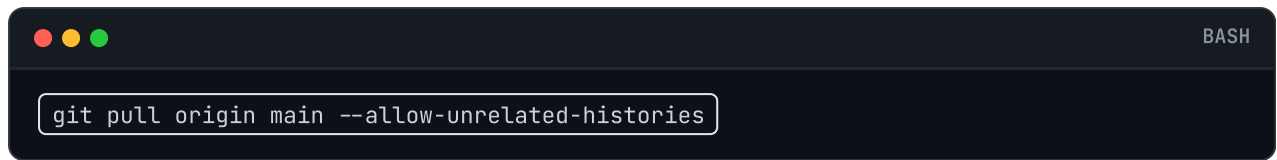
```

eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
ssh -vT git@github.com

```

2. fatal: refusing to merge unrelated histories

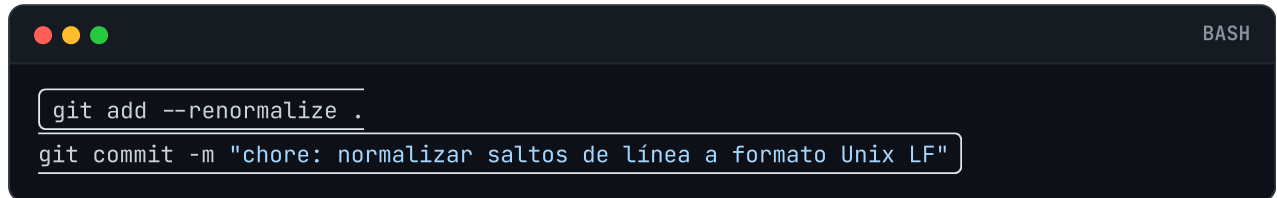
- **Solución:** Ocurre al intentar unir dos repositorios creados de forma independiente:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "BASH" is in the top-right corner. A command is entered in a light-colored box: `git pull origin main --allow-unrelated-histories`.

```
git pull origin main --allow-unrelated-histories
```

3. Normalizar saltos de línea de forma retroactiva (CRLF a LF)

- Solución:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "BASH" is in the top-right corner. Two commands are entered in light-colored boxes: `git add --renormalize .` and `git commit -m "chore: normalizar saltos de línea a formato Unix LF"`.

```
git add --renormalize .  
git commit -m "chore: normalizar saltos de línea a formato Unix LF"
```